

Django REST Framework – Conceptual Assignment

Q1. Explain why REST APIs are preferred for mobile and frontend-heavy apps.

Ans. REST APIs are widely used in mobile and frontend-heavy applications because they are lightweight and easy to integrate. They use standard HTTP methods like GET, POST, PUT, and DELETE, which makes communication simple between client and server. REST APIs usually return data in JSON format, which is easy for frontend frameworks like React or mobile apps to understand. They are also stateless, meaning each request is independent, which improves scalability. Developers can build backend and frontend separately, which speeds up development. Overall, REST APIs provide flexibility and better performance for modern applications.

Example: Example: A mobile app fetches doctor data using a REST API endpoint like `/api/doctors/` and displays it in JSON format.

Q2. Explain how serializers act as a validation layer and how to implement custom field-level validation.

Ans. Serializers in Django REST Framework play an important role as a validation layer between incoming data and database models. They convert complex data like querysets into JSON format and also validate input data before saving. Developers can define rules inside serializers to check if data is correct. Custom field-level validation can be added using methods like `validate_fieldname`. This helps ensure that only valid data is stored in the database. It also reduces errors and improves application reliability. Serializers make data handling simple and structured in APIs.

Example: Example: If a name field should not be empty, we can write `validate_name()` in serializer to check it before saving.

Q3. Explain the importance of using appropriate HTTP status codes (201 Created vs 200 OK) for API outcomes.

Ans. HTTP status codes help the client understand the result of an API request. Using correct codes improves communication between frontend and backend. For example, 200 OK means the request was successful, while 201 Created means a new resource has been successfully created. This difference is important when working with APIs. Proper status codes also help in debugging and error handling. They make the API more professional and standard. Using correct codes ensures better clarity and user experience for developers and applications.

Example: Example: When creating a doctor using POST request, API returns 201 Created instead of 200 OK.

Q4. Explain why pagination is required when listing doctors and how it impacts database performance.

Ans. Pagination is important when dealing with large amounts of data, such as listing many doctors in an API. Instead of sending all records at once, pagination divides data into smaller pages. This reduces load on the server and improves performance. It also decreases response time and avoids memory issues. For users, it becomes easier to navigate data page by page. Without pagination, the system can become slow and inefficient. So, pagination helps in better performance and user experience.

Example: Example: Instead of showing 1000 doctors at once, API shows 10 doctors per page using pagination.

Q5. Explain the benefits of ViewSets over APIView for rapid CRUD development.

Ans. ViewSets in Django REST Framework provide a faster way to build CRUD operations compared to APIView. Instead of writing separate methods for each operation, ViewSets combine all logic in one class. They automatically handle common actions like create, retrieve, update, and delete. When used with routers, URLs are also generated automatically. This reduces code length and saves development time. It

also makes the code cleaner and easier to maintain. ViewSets are very useful for rapid API development.

Example: Example: Using ModelViewSet automatically gives endpoints like GET /doctors/ and POST /doctors/.

Q6. Explain the use of Atomic Transactions (transaction.atomic) when creating related doctor records to ensure data integrity.

Ans. Atomic transactions are used to ensure that database operations are completed fully or not at all. When creating related doctor records, there can be multiple database operations involved. If one operation fails, it can lead to incomplete or incorrect data. Using transaction.atomic ensures that all operations are treated as a single unit. If any error occurs, all changes are rolled back. This maintains data integrity and prevents corruption. It is very useful in critical operations where consistency is important.

Example: Example: While saving doctor and related profile, if one fails, transaction.atomic rolls back both saves.